```
assert.ok 4 is 4
# undefined
```

If the assertion fails, an exception is thrown:

```
assert.ok 4 == 5
# AssertionError: false == true
```

Tests *describe* something the program does. The description is an important part of the test. One way to put a description in a test is by wrapping it in a function assigned to a variable named after the description:

```
do assert4Equals4 = ->
  assert.ok 4 == 4
```

Use the do -> to invoke the test function immediately.

Another useful assertion method is `assert.equal`, which takes two arguments and throws an exception if the arguments aren't equal:

```
do assert4Equals4 = ->
  assert.equal 4, 4
```

Although obviously fake, this small test demonstrates a very basic *unit test*. A test to see if a small unit of your program does what it should do is a unit test.

10.2.2. How to unit test

In order to understand the mechanics of writing tests, it's worthwhile to take only a small part of an application and implement it using a test-driven approach. Suppose that as part of the dashboard for Agtron's tracking application, you need to highlight the most popular product. To highlight it you must add a class attribute to a DOM element in a web page. Consider the following HTML fragment:

```
<html>
<div class="product special">X12</div>
```

The `<div>` has two classes: `product` and `special`. You want some way to add a third class of `popular` so that it becomes